

ACTIVIDAD DE APRENDIZAJE 3

Rey David Borrego Pérez

7502410022

Desarrollo de Software Web

Ingeniería de Software

Universidad de Cartagena

2026-1



Resumen

El presente trabajo describe el diseño e implementación de una API RESTful desarrollada con Spring Boot, Spring Web, Spring Data JPA, H2 y Swagger/OpenAPI. El proyecto aplica una estructura basada en arquitectura hexagonal y principios de Domain Driven Design (DDD), con el propósito de separar el núcleo del dominio de los detalles técnicos de infraestructura y exposición HTTP. La solución contempla módulos funcionales de gestión de usuarios y pedidos de restaurante, con operaciones CRUD completas mediante endpoints REST. Además, incorpora DTO de entrada y salida, mapeadores, puertos de entrada y salida, adaptadores de persistencia, manejo global de excepciones y documentación de servicios. El control de versiones fue planteado mediante Git y GitHub con ramas por funcionalidad, commits progresivos y documentación en README. Los resultados demuestran la viabilidad de aplicar patrones arquitectónicos modernos en un backend académico orientado al consumo por clientes web o herramientas HTTP.

1. Introducción

Las aplicaciones web actuales suelen estar compuestas por clientes frontend, servicios backend y bases de datos que interactúan mediante protocolos y contratos definidos. En este escenario, las API RESTful cumplen un papel esencial porque permiten exponer operaciones del sistema a través de endpoints HTTP, facilitando la integración con diferentes interfaces, aplicaciones móviles, clientes web o herramientas de prueba como Postman y Swagger.

El presente proyecto fue desarrollado en el marco de la asignatura Desarrollo de Software Web, correspondiente al programa de Ingeniería de Software de la Universidad de Cartagena. Su propósito central es construir una API RESTful funcional que permita gestionar usuarios y pedidos de restaurante, aplicando arquitectura hexagonal y principios de diseño guiado por el dominio para lograr una estructura modular, desacoplada y mantenible.

La actividad representa una evolución frente al enfoque MVC de la unidad anterior, ya que en este caso la interacción no se realiza mediante vistas HTML renderizadas por el servidor, sino por medio de servicios REST que retornan datos estructurados. Este cambio permite comprender la diferencia entre una aplicación web monolítica con vistas y un backend preparado para ser consumido por clientes externos.

Repositorio del proyecto: <https://github.com/ReyDavid07/unidad3-spring-hexagonal-rest.git>

2. Marco Teórico

2.1. API RESTful

Una API RESTful es una interfaz de programación que expone recursos mediante rutas HTTP y utiliza métodos como GET, POST, PUT y DELETE para consultar o modificar el estado del sistema. Este estilo de comunicación favorece la interoperabilidad porque los clientes no necesitan conocer la implementación interna del backend, sino únicamente los endpoints disponibles, los datos requeridos y las respuestas esperadas.

En este proyecto, los recursos principales son usuarios y pedidos. Cada recurso cuenta con endpoints para crear, listar, consultar por identificador, actualizar y eliminar registros. De esta manera, la API puede ser consumida por herramientas HTTP, por un frontend desarrollado en cualquier framework o por servicios externos.

2.2. Arquitectura Hexagonal

La arquitectura hexagonal, también conocida como puertos y adaptadores, propone aislar el núcleo de la aplicación de los detalles técnicos externos. El dominio contiene las reglas y modelos principales; los puertos definen contratos de entrada y salida; y los adaptadores implementan la interacción con tecnologías específicas, como controladores REST o repositorios JPA.

En la solución desarrollada, esta arquitectura se evidencia en la separación entre domain, application e infrastructure. El dominio define los modelos y puertos; la capa de aplicación implementa los servicios que ejecutan los casos de uso; y la infraestructura contiene los controladores REST, las entidades JPA, los repositorios concretos, los mapeadores y la configuración técnica.

2.3. Domain Driven Design (DDD)

El Domain Driven Design es un enfoque de diseño que propone modelar el software a partir del dominio del problema. Bajo este enfoque, las entidades, reglas, conceptos y operaciones del negocio deben reflejarse de forma clara en el código fuente. En este proyecto, las entidades

Usuario y Pedido se ubican en el dominio, permitiendo que el sistema exprese los conceptos principales del caso académico.

La aplicación de DDD en este backend facilita la comprensión del sistema porque diferencia los objetos propios del negocio de los objetos técnicos utilizados para recibir solicitudes HTTP o almacenar información en la base de datos. Por ello, se emplean DTO para la comunicación REST y entidades JPA para la persistencia, evitando mezclar responsabilidades.

2.4. DTO, mapeadores y manejo de errores

Los DTO, o Data Transfer Objects, son clases utilizadas para transportar datos entre capas o entre el cliente y el servidor. Su uso permite controlar qué información ingresa o sale de la API sin exponer directamente los modelos de dominio o las entidades de persistencia. En el proyecto se definieron DTO de creación, actualización y respuesta para usuarios y pedidos.

Los mapeadores permiten convertir objetos entre representaciones, por ejemplo de DTO a modelo de dominio o de entidad JPA a respuesta REST. Además, el manejo global de excepciones permite responder de forma organizada ante errores como registros no encontrados o datos inválidos, mejorando la claridad del backend para el cliente que consume la API.

2.5. Control de Versiones con Git y GitHub

Git permite registrar los cambios del código fuente y GitHub facilita el alojamiento remoto del proyecto. En una actividad de backend REST, el historial de commits es especialmente importante porque permite evidenciar la evolución de la arquitectura: creación del dominio, definición de puertos, implementación de servicios, construcción de adaptadores y documentación de endpoints.

3. Objetivos

3.1. Objetivo General

Desarrollar una API RESTful con Spring Boot basada en arquitectura hexagonal y principios DDD, que permita gestionar usuarios y pedidos de restaurante mediante operaciones CRUD, documentación de endpoints, manejo de errores y control de versiones en GitHub.

3.2. Objetivos Específicos

- Diseñar una estructura de proyecto que separe dominio, aplicación e infraestructura siguiendo el enfoque de puertos y adaptadores.
- Implementar modelos de dominio para usuarios y pedidos, evitando que la lógica principal dependa de controladores o entidades de persistencia.
- Crear endpoints REST para listar, consultar, registrar, actualizar y eliminar usuarios y pedidos.
- Utilizar DTO y mapeadores para controlar el intercambio de datos entre el cliente HTTP y el backend.
- Configurar Swagger/OpenAPI y un archivo requests.http para facilitar la documentación y prueba de los servicios.
- Gestionar el proceso de desarrollo mediante ramas, commits progresivos, README y repositorio remoto en GitHub.

4. Tecnologías Utilizadas

La selección de tecnologías se orientó a construir una API REST ejecutable de manera local, documentada y organizada bajo buenas prácticas de ingeniería de software. Las herramientas principales se describen a continuación.

4.1. Java y Spring Boot

Java fue utilizado como lenguaje principal por su orientación a objetos y su integración con el ecosistema Spring. Spring Boot simplificó la configuración del proyecto, el manejo de dependencias y la ejecución del servidor embebido necesario para exponer los endpoints REST.

4.2. Spring Web y servicios REST

Spring Web permitió crear controladores REST mediante anotaciones como `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping` y `@DeleteMapping`. Estas anotaciones facilitan la definición de rutas HTTP y la asociación de cada operación con un método del controlador.

4.3. Spring Data JPA y H2

Spring Data JPA se utilizó para implementar repositorios de persistencia a partir de interfaces JPA, mientras que H2 sirvió como base de datos en memoria para pruebas locales. Esta combinación permite ejecutar el proyecto sin instalar un gestor externo, manteniendo una estructura realista de persistencia.

4.4. Swagger/OpenAPI

Swagger/OpenAPI se integró para generar documentación interactiva de la API. Esta herramienta permite visualizar los endpoints disponibles, los métodos HTTP, los parámetros requeridos y los esquemas de datos, además de facilitar pruebas directamente desde el navegador.

4.5. Maven, Git y GitHub

Maven administró las dependencias y el ciclo de ejecución del proyecto. Git permitió controlar los cambios del código fuente y GitHub funcionó como repositorio remoto para presentar el desarrollo, el historial de commits y la documentación requerida por el tutor.

5. Desarrollo del Sistema

5.1. Estructura General del Proyecto

El proyecto se organizó en tres zonas principales: domain, application e infrastructure. Esta estructura permite que las reglas del negocio permanezcan separadas de los detalles de exposición REST y persistencia. La capa de dominio contiene los modelos, excepciones y puertos; la capa de aplicación implementa los servicios; y la infraestructura contiene controladores, DTO, mapeadores, adaptadores de persistencia y configuraciones.

Capa	Elementos principales	Responsabilidad
Dominio	model, port, exception	Define entidades, contratos y excepciones propias del negocio.
Aplicación	service	Implementa casos de uso y coordina operaciones entre puertos.
Infraestructura	controller, dto, mapper, adapter, config	Conecta el dominio con HTTP, JPA, Swagger y configuración técnica.

5.2. Módulo de Gestión de Usuarios

El módulo de usuarios permite administrar registros asociados a nombre, correo electrónico, contraseña, rol y estado. Las operaciones se exponen mediante endpoints REST que reciben y retornan datos en formato JSON. Para mantener la separación arquitectónica, el controlador no accede directamente al repositorio JPA, sino que invoca un caso de uso representado por el puerto de entrada.

El flujo inicia en el controlador REST, continúa con el mapeo del DTO al modelo de dominio, pasa por el servicio de aplicación y finaliza en el adaptador de persistencia, que implementa el puerto de salida. La respuesta se construye nuevamente mediante un mapeador para retornar un DTO de salida al cliente.

5.3. Módulo de Gestión de Pedidos de Restaurante

El módulo de pedidos representa el caso asignado de restaurante. La entidad Pedido contiene datos como número de pedido, cliente, producto, cantidad, precio total y estado. Al igual que el módulo de usuarios, la gestión de pedidos se implementó con operaciones de creación, listado, consulta por identificador, actualización y eliminación.

Este módulo demuestra la reutilización del patrón arquitectónico en un segundo recurso de negocio. La existencia de puertos, servicios, adaptadores y mapeadores para pedidos permite mantener una estructura uniforme y facilita la extensión futura del sistema, por ejemplo incorporando filtros por estado o cálculo automático de valores.

5.4. Endpoints REST implementados

La API expone endpoints para dos recursos principales: usuarios y pedidos. Cada recurso cuenta con operaciones equivalentes que siguen la convención REST para creación, consulta, actualización y eliminación de registros.

Método	Endpoint	Descripción
POST	/api/users	Crear un usuario.
GET	/api/users	Listar todos los usuarios.
GET	/api/users/{id}	Consultar un usuario por identificador.
PUT	/api/users/{id}	Actualizar los datos de un usuario.
DELETE	/api/users/{id}	Eliminar un usuario.

POST	/api/pedidos	Crear un pedido de restaurante.
GET	/api/pedidos	Listar todos los pedidos.
GET	/api/pedidos/{id}	Consultar un pedido por identificador.
PUT	/api/pedidos/{id}	Actualizar los datos de un pedido.
DELETE	/api/pedidos/{id}	Eliminar un pedido.

5.5. Documentación y prueba de la API

Para facilitar la prueba de la API, el proyecto incluye configuración de Swagger/OpenAPI y un archivo `requests.http`. Swagger permite revisar los endpoints desde el navegador en la ruta `/swagger-ui.html`, mientras que el archivo `requests.http` permite ejecutar peticiones desde Visual Studio Code con la extensión REST Client.

La base de datos H2 se carga durante la ejecución local, lo cual permite contar con datos iniciales de prueba y validar rápidamente las operaciones de consulta, creación, actualización y eliminación. Esta estrategia reduce la complejidad de instalación y facilita la revisión por parte del tutor.

6. Pruebas Realizadas

La verificación del sistema se planteó mediante pruebas funcionales manuales sobre los endpoints REST. Aunque el entorno de generación del informe no cuenta con Maven para ejecutar el proyecto, el backend queda preparado para ser probado en un equipo local con Java 17 y Maven mediante el comando `mvn spring-boot:run`.

- Ejecución del backend y acceso a Swagger UI en `http://localhost:8080/swagger-ui.html`.
- Listado de usuarios mediante GET `/api/users` y consulta individual mediante GET `/api/users/{id}`.
- Creación de usuarios mediante POST `/api/users` con cuerpo JSON válido.
- Actualización de usuarios mediante PUT `/api/users/{id}`.
- Eliminación de usuarios mediante DELETE `/api/users/{id}`.
- Listado, creación, consulta, actualización y eliminación de pedidos mediante los endpoints `/api/pedidos`.
- Validación de respuestas ante identificadores inexistentes, verificando el manejo de errores.

- Revisión de la consola H2 para confirmar la persistencia de datos durante la ejecución.

Estas pruebas permiten confirmar que la API cumple con las operaciones CRUD requeridas. Como mejora futura, se recomienda incorporar pruebas automatizadas unitarias y de integración para validar servicios, controladores y adaptadores de persistencia.

7. Control de Versiones

El proceso de desarrollo debe evidenciarse mediante Git y GitHub. Para cumplir con los criterios de la actividad, se recomienda subir el proyecto con un historial compuesto por commits progresivos que demuestren la construcción de cada capa de la arquitectura.

- Rama principal (main): debe contener la versión estable y funcional del backend REST.
- Ramas de funcionalidad: pueden utilizarse ramas como feature/domain-models, feature/application-services, feature/persistence-adapters, feature/rest-controllers y feature/swagger-documentation.
- Commits progresivos: se recomienda registrar avances separados para dominio, puertos, servicios, persistencia, DTO, controladores, manejo de errores, documentación y README.
- Pull Requests: si se desea demostrar un flujo más profesional, cada rama puede integrarse a main mediante Pull Request en GitHub.

El uso de ramas y commits organizados permite demostrar que el proyecto no fue construido en un único paso, sino mediante un proceso trazable y coherente con las prácticas de desarrollo colaborativo.

8. Conclusiones

El desarrollo de la API REST permitió comprender la importancia de separar la exposición HTTP de la lógica del dominio. A diferencia de una aplicación MVC con vistas, este backend está preparado para ser consumido por distintos clientes, lo cual amplía las posibilidades de integración del sistema.

En segundo lugar, la arquitectura hexagonal aportó claridad estructural al proyecto. Al ubicar el dominio en el centro y rodearlo de puertos y adaptadores, se reduce el acoplamiento con tecnologías específicas como JPA, Swagger o los controladores REST.

En tercer lugar, el uso de DTO y mapeadores permitió proteger los modelos internos del sistema y controlar de manera explícita la información que entra y sale de la API. Esta práctica mejora la mantenibilidad y evita que los cambios en una capa afecten directamente a las demás.

Finalmente, Git y GitHub constituyen herramientas indispensables para evidenciar el proceso de desarrollo. Un historial organizado por ramas y commits facilita la revisión del tutor, documenta las decisiones técnicas y aproxima el trabajo académico a prácticas profesionales. Como trabajo futuro, se propone implementar seguridad con autenticación, pruebas automatizadas, paginación y despliegue del backend en un servicio en la nube.

Referencias

Chacon, S., y Straub, B. (2014). Pro Git (2.a ed.). Apress. <https://git-scm.com/book/en/v2>

Cockburn, A. (2005). Hexagonal architecture. Alistair Cockburn.
<https://alistair.cockburn.us/hexagonal-architecture/>

Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley Professional.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation). University of California, Irvine.

Spring. (2024). Spring Boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Springdoc. (2024). Springdoc OpenAPI documentation. <https://springdoc.org/>